Disha Kulkarni roll no 38 D15B

EXPERIMENT 1
Implementation of Linear and Logistic Regression on Real-World Dataset

Aim :
To study the concept of regression and to implement Linear Regression and Logistic Regression models on a real-world student performance dataset for prediction and classification purposes.

Theory

What is Regression?

Regression is a supervised machine learning technique used to predict output values based on input features. In regression, the model learns a relationship between independent variables (features) and a dependent variable (target) using labeled data.

Based on the nature of the output variable, regression techniques are mainly classified into:

- Linear Regression → Used to predict continuous numerical values
- Logistic Regression → Used to predict binary or categorical outcomes (0 or 1)

Introduction to Regression

Regression is a type of supervised learning where a model is trained on historical data containing both input features and corresponding output values. The goal is to learn the underlying relationship between inputs and outputs so that accurate predictions can be made on unseen data.

Regression models are widely used in real-world applications such as price prediction, risk assessment, medical diagnosis, and performance analysis.

Linear Regression

Overview

Linear Regression models the relationship between input variables and a continuous output variable by fitting a straight line that best represents the data points. The objective is to minimize the difference between the predicted values and the actual values.

Mathematical Representation

For a single input variable:

$$y = wx + b$$

For multiple input variables:

$$y = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Where:

- y represents the predicted output
- $x_1, x_2, \ldots, x_n$ represent input features
- $w_1, w_2, \ldots, w_n$ represent learned weights
- b represents the bias or intercept

Linear Regression Model Formulation

For multiple predictors, Linear Regression can also be written as:

$$y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \cdots + \beta_nx_n$$

Where:

- β_0 is the intercept
- $\beta_1, \beta_2, \ldots, \beta_n$ are regression coefficients

Logistic Regression

Overview

Logistic Regression is a classification algorithm used when the output variable has only two possible outcomes. Instead of predicting exact values, it predicts the probability that a given input belongs to a particular class.

Sigmoid Function

Logistic Regression uses the sigmoid (logistic) function to map predicted values between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The output of the sigmoid function is interpreted as the probability of the data point belonging to a specific class.

Dataset Description

Dataset Source

- Dataset Name: Student Performance Factors Dataset
- File Name: `House Price Prediction.csv`
- Source: Kaggle

Dataset Summary

The dataset contains academic attributes related to students and their influence on final examination performance.

- Approximate Records: 1000 students
- Data Format: Structured and tabular
- Application Domain: Education and academic analytics

Attributes Used

Independent Variables

- Hours_Studied – Time spent studying
- Attendance – Attendance percentage
- Assignment_Score – Assignment performance
- Midterm_Score – Midterm examination marks

Dependent Variable

- Final_Score – Final examination marks

Since *Final_Score* is a continuous numerical value, it is suitable for Linear Regression.

For Logistic Regression, *Final_Score* is converted into a binary class:

- Score ≥ 65 → Pass (1)
- Score < 65 → Fail (0)

Limitations of Linear Regression

- Assumes a linear relationship between variables
- Highly sensitive to outliers

- Cannot model complex or non-linear patterns
- Performance degrades with multicollinearity
- Requires numerical input features

Experimental Procedure

1. The dataset was loaded into the system.
2. Missing values were removed to ensure data consistency.
3. The dataset was divided into input features and output labels.
4. Data was split into training and testing sets in an 80:20 ratio.
5. Linear Regression was trained to predict final scores.
6. Logistic Regression was trained to classify outcomes.
7. Model predictions were evaluated using appropriate metrics.

Performance Evaluation

Linear Regression Metrics

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- R^2 Score

A high R^2 value close to 1 indicates strong predictive performance.

Logistic Regression Metrics

- Accuracy
- Confusion Matrix
- Precision, Recall, and F1-Score

Hyperparameter Optimization

Linear Regression has limited tunable parameters, but its performance may vary depending on factors such as intercept handling and normalization.

For Logistic Regression, GridSearchCV was used to:

- Test different values of the regularization parameter C
- Experiment with multiple solver algorithms
- Select the best model using cross-validation

Tools and Libraries Used

- NumPy – Numerical computations

- Pandas – Data handling and preprocessing
- Matplotlib – Data visualization
- Seaborn – Statistical visualization
- Scikit-learn – Model training, evaluation, and hyperparameter tuning

Code:

```
# ==============================
# STEP 1: Import Libraries
# ==============================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    accuracy_score,
    confusion_matrix,
    classification_report
)

# ==============================
# STEP 2: Load Dataset
# ==============================
data = pd.read_csv("House Price Prediction Dataset.csv")
print(data.head())

# ==============================
# STEP 3: Data Cleaning
# ==============================
data = data.dropna()

# ==============================
# STEP 4: Encode Categorical Columns
# ==============================
le = LabelEncoder()
```

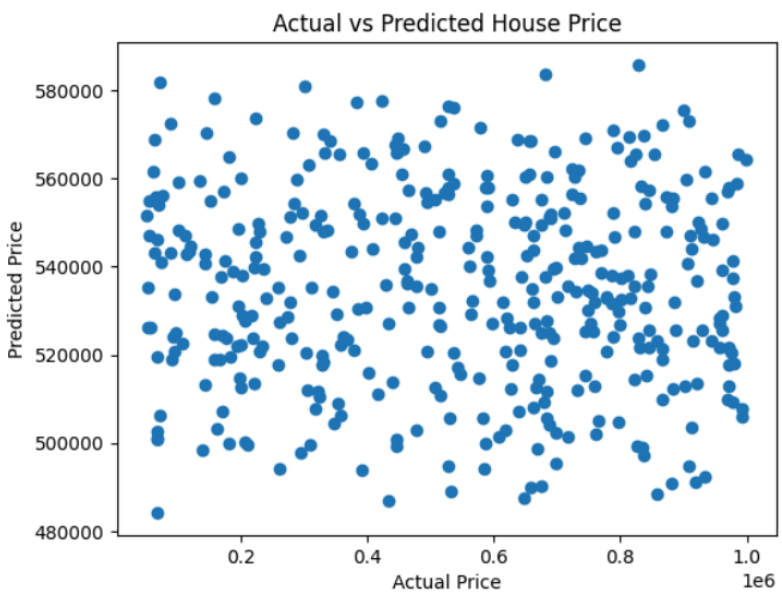
```
data["Location"] = le.fit_transform(data["Location"])
data["Condition"] = le.fit_transform(data["Condition"])
data["Garage"] = le.fit_transform(data["Garage"])

# ==============================
# PART A: LINEAR REGRESSION
# ==============================
X = data.drop(columns=["Price", "Id"])
y = data["Price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)

print("\n===== LINEAR REGRESSION RESULTS =====")
print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score:", r2_score(y_test, y_pred))

plt.scatter(y_test, y_pred)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted House Price")
plt.show()

# ==============================
# PART B: LOGISTIC REGRESSION
# ==============================
# Create binary target (Expensive / Not Expensive)
data["Expensive"] = data["Price"].apply(lambda x: 1 if x >=
data["Price"].median() else 0)

X = data.drop(columns=["Price", "Expensive", "Id"])
y = data["Expensive"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)

y_pred_log = log_model.predict(X_test)

print("\n===== LOGISTIC REGRESSION RESULTS =====")
print("Accuracy:", accuracy_score(y_test, y_pred_log))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_log))
print("Classification Report:\n", classification_report(y_test, y_pred_log))

sns.heatmap(confusion_matrix(y_test, y_pred_log), annot=True, fmt="d")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# ==============================
# PART C: HYPERPARAMETER TUNING
# ==============================
param_grid = {
    "C": [0.01, 0.1, 1, 10],
    "solver": ["liblinear", "lbfgs"]
}

grid = GridSearchCV(
    LogisticRegression(max_iter=1000),
    param_grid,
    cv=5
)

grid.fit(X_train, y_train)

print("\n===== HYPERPARAMETER TUNING =====")
print("Best Parameters:", grid.best_params_)
print("Best Cross Validation Accuracy:", grid.best_score_)
```

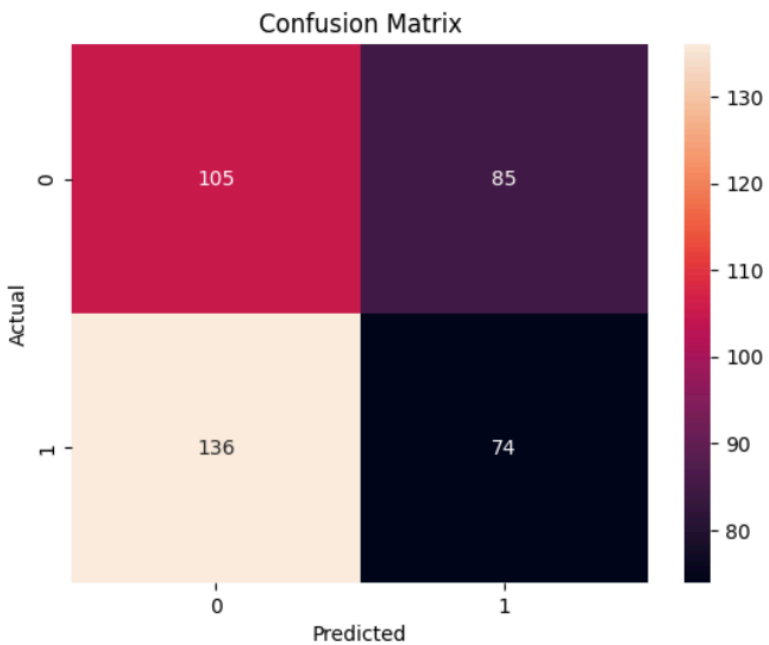
Screenshots :

Disha Kulkarni roll no 38 D15B

```
     Id  Area  Bedrooms  Bathrooms  Floors  YearBuilt  Location  Condition  \
0   1  1360         5          4       3       1970  Downtown  Excellent
1   2  4272         5          4       3       1958  Downtown  Excellent
2   3  3592         2          2       3       1938  Downtown       Good
3   4   966         4          2       2       1902  Suburban       Fair
4   5  4926         1          4       2       1975  Downtown       Fair

  Garage   Price
0     No  149919
1     No  424998
2     No  266746
3    Yes  244020
4    Yes  636056


===== LINEAR REGRESSION RESULTS =====
MAE: 244419.99901363705
MSE: 78729422262.64821
RMSE: 280587.637401665
R2 Score: -0.011961539273188437
```



```
===== LOGISTIC REGRESSION RESULTS =====
Accuracy: 0.4475
Confusion Matrix:
 [[105  85]
 [136  74]]
Classification Report:
              precision    recall  f1-score   support

           0       0.44      0.55      0.49       190
           1       0.47      0.35      0.40       210

    accuracy                           0.45       400
   macro avg       0.45      0.45      0.44       400
weighted avg       0.45      0.45      0.44       400
```




```
===== HYPERPARAMETER TUNING =====
Best Parameters: {'C': 0.01, 'solver': 'liblinear'}
Best Cross Validation Accuracy: 0.5149999999999999
```

Conclusion:

This experiment demonstrated the application of Linear and Logistic Regression techniques for prediction and classification tasks. Linear Regression effectively predicted continuous output values, while Logistic Regression accurately classified data into binary categories. Model performance was evaluated using appropriate metrics, and hyperparameter tuning improved classification accuracy. The experiment highlights the importance of regression models in supervised machine learning and real-world data analysis.